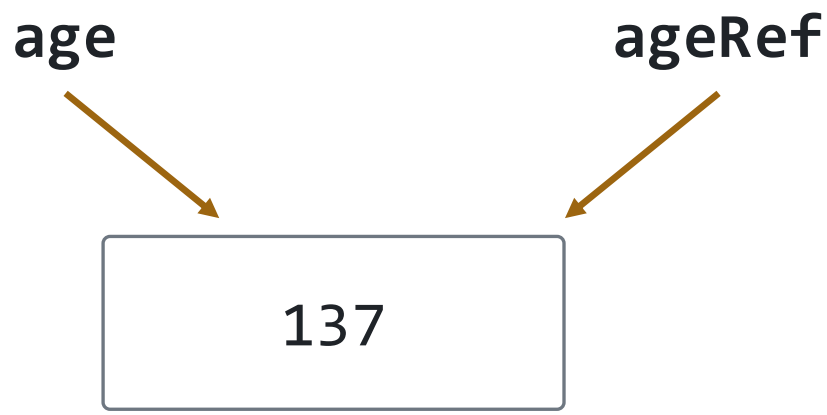
Pointers vs. references

The short version

A **reference** is *another name* for a variable that already exists.

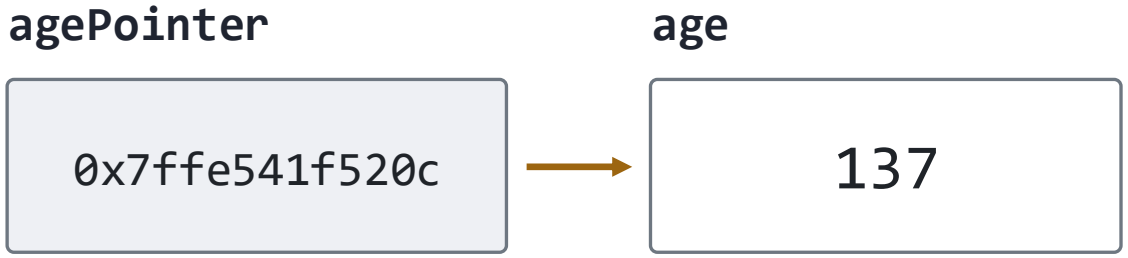
```
int &ageRef = age;
```



one `int`, two names

A **pointer** is *a separate variable* that holds the **address** of another variable.

```
int *agePointer = &age;
```



its own box: an address

These do the same thing

With references

```
4   void swapInts(int &a, int &b) {
5       int temp = a;
6       a = b;
7       b = temp;
8   }
9
10  int main() {
11      int x = 1;
12      int y = 2;
13      swapInts(x, y);
14      cout << x << " " << y << endl;
15      return 0;
16  }
```

With pointers

```
4   void swapInts(int *a, int *b) {
5       int temp = *a;
6       *a = *b;
7       *b = temp;
8   }
9
10  int main() {
11      int x = 1;
12      int y = 2;
13      swapInts(&x, &y);
14      cout << x << " " << y << endl;
15      return 0;
16  }
```

Both print `2 1`. The reference version reads like ordinary code. The pointer version makes the caller write `&x`, and the function write `*a` every time.

How they differ

	Reference: `int &r = x;`	**Pointer: `int *p = &x;`**
What is it?	Another name for `x`	A variable holding `x`'s address
Must it be set up right away?	Yes: `int &r;` won't compile	No, but always initialize it
Can it be "nothing"?	No	Yes: `nullptr`
Can it switch to another variable?	No, never	Yes: `p = &y;`
How do you reach `x`?	Just use `r`	`*p`, or `p->` for members
Can it hold what `new` returns?	No	Yes

`r = best;` vs. `p = &best;`

Assigning to a reference copies a value

```
4    int main() {
5        int score = 10;
6        int best = 99;
7        int &r = score;
8
9        r = best;
10       cout << score << " " << best << endl;
11
12       return 0;
13   }
```

```
$ ./refMove
99 99
```

Assigning to a pointer moves it

```
4    int main() {
5        int score = 10;
6        int best = 99;
7        int *p = &score;
8
9        p = &best;
10       *p = 0;
11       cout << score << " " << best << endl;
12
13       return 0;
14   }
```

```
$ ./ptrMove
10 0
```

A reference is stuck to its variable forever: `r = best;` puts 99 *into* `score`. A pointer can move: `p = &best;` points it somewhere new.

A little history

Early 1970s: C

Only pointers. To let a function change your variable, you pass its address: `swap(&x, &y)`.

Around 1980: C++

Stroustrup adds references, mainly so operators can work on objects: `a + b`, not `&a + &b`.

Why `this` is a pointer

`this` was in the language before references existed, so it stayed a pointer.

2011: `nullptr`

C++11 adds `nullptr`. Older code and older books use `NULL` or `0`.

C++ almost never removes old features. It adds new ones next to them. That's why you'll see pointers, references, `NULL`, and `nullptr` all in real code.

Why we still need pointers

Mean “nothing”

A pointer can be `nullptr`. Useful when “there's nothing here” is a real answer.

Point somewhere new

`p = &y;` Next time: walking through an array one element at a time.

Reach heap objects

`new` hands back an address. Only a pointer can hold it.

Talk to C code

C libraries expect pointers, because pointers are all C has.

References handle the most common job. Pointers handle everything references can't.

Which one should I use?

If…	Use…
a function needs to change the caller's variable	a reference: `void addTen(int &n)`
the function might be handed "nothing"	a pointer, and check for `nullptr` first
it needs to point at different things over time	a pointer
you used `new`	a pointer, and exactly one `delete` later

Either way: **always initialize pointers**, and set them to `nullptr` after `delete`.

Practice

pointers and references practice sheet